PgBouncer for PostgreSQL

Installation, Configuration, Operation and Monitoring - Step by Step

Item	Value used in this guide
PostgreSQL	16 or later, listening on port 5432
PgBouncer	1.21 or later recommended (prepared statements in transaction mode)
PgBouncer port	6432 (default)
OS	Ubuntu/Debian (RHEL/Rocky differences noted)
Example database / user	appdb / appuser

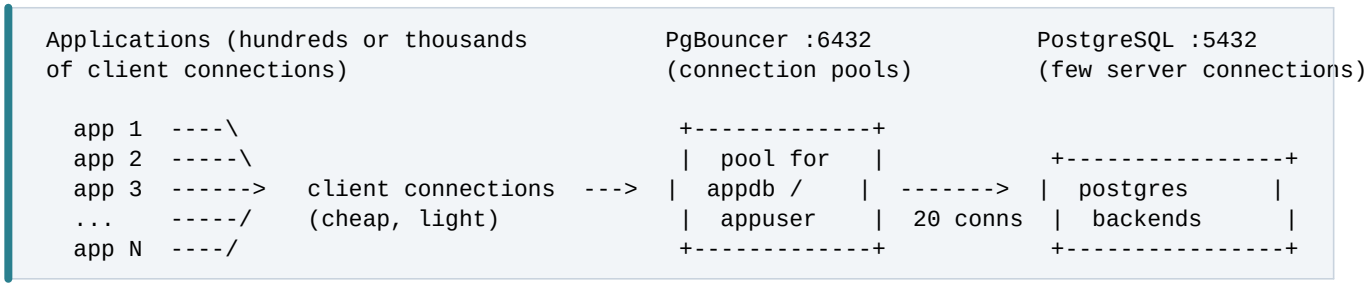
Important: Parameter names and defaults are taken from the PgBouncer documentation. Run `pgbouncer --version` and check pgbouncer.org/config.html for your exact version before applying settings in production.

Contents

- 1. What PgBouncer is and why you need it
- 2. Pooling modes and which one to choose
- 3. Installation
- 4. Prepare PostgreSQL (roles, auth function, pg_hba.conf)
- 5. Configure PgBouncer (pgbouncer.ini and userlist.txt)
- 6. Start, test and connect applications
- 7. Sizing the pools
- 8. Administration (admin console and operations)
- 9. Monitoring and alerting
- 10. Security (TLS, access control)
- 11. High availability and failover
- 12. Application compatibility in transaction mode
- 13. Troubleshooting and cheat sheet

1. What PgBouncer Is and Why You Need It

PgBouncer is a lightweight connection pooler for PostgreSQL. Applications connect to PgBouncer instead of PostgreSQL. PgBouncer keeps a small number of real server connections open and shares them among many client connections.



Why this matters. Every PostgreSQL connection is a separate server process using memory (several MB) and scheduling resources. Thousands of mostly idle connections waste memory and slow down the server, and opening a new connection is expensive (process fork, authentication, TLS). A pooler solves these problems:

- **Fewer backend processes:** 2000 application connections can be served by 20 to 50 PostgreSQL backends.
- **Faster connects:** clients connect to a long-running pooler; server connections are reused.
- **Protection:** limits per database and per user prevent connection storms from overloading PostgreSQL.
- **Operational flexibility:** PAUSE/RESUME and backend host changes allow maintenance without dropping clients.
- **Low overhead:** PgBouncer is single-threaded, uses very little memory per client, and is not an SQL proxy (it does not parse queries).

When PgBouncer is the right tool

- Many short-lived connections (web apps, serverless, microservices).
- Many application instances each holding their own connection pool, giving a high total connection count.
- You are hitting `max_connections` or see high memory use from idle backends.

It is less useful for a handful of long-lived connections that already stay busy.

2. Pooling Modes

Mode	Server connection is released	Pros	Cons / restrictions
session	When the client disconnects	Fully compatible with every PostgreSQL feature	Lowest connection reuse. Idle clients still hold server connections.
transaction	At the end of each transaction	Best reuse; most common production choice	Session features do not work reliably (see below).
statement	After every statement	Maximum reuse	Multi-statement transactions are not allowed. Rarely used.

Features that break or need care in transaction mode

Feature	What to do
SET / RESET of session variables	Use `SET LOCAL` inside a transaction, or set defaults per role/database with ALTER ROLE / ALTER DATABASE.
Session-level advisory locks	Use `pg_advisory_xact_lock` (transaction scope) instead.
LISTEN / NOTIFY	Needs a session-mode connection. Use a separate database alias in session mode.
WITH HOLD cursors, temporary tables across transactions	Keep the work inside one transaction, or use session mode.
Protocol-level prepared statements	Supported from PgBouncer 1.21 when `max_prepared_statements` is greater than 0. On older versions, disable server-side prepares in the driver.
SQL `PREPARE` / `DEALLOCATE` statements	Not tracked by PgBouncer; avoid in transaction mode.
pg_dump, logical replication, replication connections	Connect directly to PostgreSQL (port 5432), not through PgBouncer.

Tip: A common design: keep the main alias in **transaction** mode for the application and add a second alias in **session** mode for tools that need session features.

3. Installation

Step 1 - Choose the host

- **Same server as PostgreSQL:** simplest; clients connect on 6432, PgBouncer connects locally.
- **Dedicated pooler host or per-application-server:** reduces load on the database server and is common in HA setups (Section 11).

Step 2 - Install the package

```
# Debian / Ubuntu
sudo apt update
sudo apt install -y pgbouncer

# RHEL / Rocky / Alma (PGDG repository enabled)
sudo dnf install -y pgbouncer

# Verify
pgbouncer --version
```

The PGDG repository usually carries a newer PgBouncer than the distribution repository. Prefer it for production.

Step 3 - Know the file locations

Item	Debian / Ubuntu	RHEL family
Main config	/etc/pgbouncer/pgbouncer.ini	/etc/pgbouncer/pgbouncer.ini
User list	/etc/pgbouncer/userlist.txt	/etc/pgbouncer/userlist.txt
Log file	/var/log/postgresql/pgbouncer.log	/var/log/pgbouncer/pgbouncer.log
Run as OS user	postgres	pgbouncer
Service name	pgbouncer	pgbouncer
Default port	6432	6432

Package layouts differ slightly between versions. Confirm with `systemctl cat pgbouncer` and `dpkg -L pgbouncer` or `rpm -ql pgbouncer`.

Step 4 - Raise the open files limit (important for many clients)

Each client and server connection uses a file descriptor. Add a systemd override:

```
sudo systemctl edit pgbouncer
# Add the following, save and exit:
[Service]
LimitNOFILE=65536

sudo systemctl daemon-reload
```

4. Prepare PostgreSQL

Step 1 - Application role and database (skip if they exist)

```
CREATE ROLE appuser WITH LOGIN PASSWORD 'StrongAppPassword';
CREATE DATABASE appdb OWNER appuser;
```

Make sure PostgreSQL stores passwords as SCRAM (default since PostgreSQL 14):

```
SHOW password_encryption;                    -- should be scram-sha-256
-- Roles created with older MD5 passwords must have their password reset after switching to SCRAM.
```

Step 2 - Create the authentication user and lookup function

With an **auth_query**, PgBouncer looks up users and password hashes in PostgreSQL, so you do not have to maintain userlist.txt for every application user. This is the recommended approach.

```
-- Run as superuser in the postgres database AND in each application database
CREATE ROLE pgbouncer WITH LOGIN PASSWORD 'StrongPoolerPassword';

CREATE OR REPLACE FUNCTION public.pgbouncer_get_auth(p_username TEXT)
RETURNS TABLE(username NAME, passwd TEXT)
LANGUAGE sql SECURITY DEFINER SET search_path = pg_catalog AS
$$
  SELECT username::NAME, passwd::TEXT FROM pg_catalog.pg_shadow WHERE username = p_username;
$$;

REVOKE ALL ON FUNCTION public.pgbouncer_get_auth(TEXT) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.pgbouncer_get_auth(TEXT) TO pgbouncer;
```

Note: Instead of creating the function in every database, you can create it once and set `auth_dbname = postgres` in pgbouncer.ini so all lookups go to that database.

Step 3 - Allow PgBouncer in pg_hba.conf

```
# TYPE  DATABASE  USER                ADDRESS          METHOD
host    all       pgbouncer,appuser   127.0.0.1/32     scram-sha-256
# If PgBouncer is on another host:
host    all       pgbouncer,appuser   10.0.0.20/32     scram-sha-256
```

```
sudo -u postgres psql -c "SELECT pg_reload_conf();"
```

PostgreSQL sees every client as coming from the PgBouncer host, so client-level IP rules must be enforced in PgBouncer (see `auth_hba_file` in Section 10).

Step 4 - Check PostgreSQL connection capacity

```
SHOW max_connections;
SHOW superuser_reserved_connections;
-- Pool sizes in PgBouncer must fit within max_connections (Section 7)
```

5. Configure PgBouncer

Step 1 - userlist.txt (minimum: the auth_user and admin users)

Format: one line per user, **"username" "password-or-hash"**. PgBouncer needs the `pgbouncer` user here so it can connect and run the auth query. Use the SCRAM verifier from PostgreSQL rather than a plain-text password:

```
sudo -u postgres psql -At -c "SELECT '\"' || rolname || '\" \"' || rolpassword || '\"' FROM pg_authid WH
  | sudo tee /etc/pgbouncer/userlist.txt

sudo chown postgres:postgres /etc/pgbouncer/userlist.txt      # RHEL: pgbouncer:pgbouncer
sudo chmod 640 /etc/pgbouncer/userlist.txt
```

Result looks like: `"pgbouncer" "SCRAM-SHA-256$4096:salt$storedkey:serverkey"`. For the admin console user, create a role in PostgreSQL with a password, add its hash the same way (or use a plain-text entry with strict file permissions).

Step 2 - pgbouncer.ini (complete working example)

```
[databases]
; alias = connection string to the real PostgreSQL server
appdb      = host=127.0.0.1 port=5432 dbname=appdb
appdb_sess = host=127.0.0.1 port=5432 dbname=appdb pool_mode=session pool_size=5
appdb_ro   = host=10.0.0.12 port=5432 dbname=appdb            ; read replica
; optional fallback: * = host=127.0.0.1 port=5432

[pgbouncer]
; --- network ---
listen_addr = 0.0.0.0
listen_port = 6432
unix_socket_dir = /var/run/postgresql

; --- authentication ---
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
auth_user = pgbouncer
auth_query = SELECT usename, passwd FROM public.pgbouncer_get_auth($1)
; auth_dbname = postgres
admin_users = pgbouncer
stats_users = pgbouncer_stats

; --- pooling ---
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 20
min_pool_size = 5
reserve_pool_size = 5
reserve_pool_timeout = 3
max_db_connections = 50
max_user_connections = 0

; --- timeouts ---
server_idle_timeout = 600
server_lifetime = 3600
server_connect_timeout = 15
query_wait_timeout = 120
client_idle_timeout = 0
client_login_timeout = 60

; --- compatibility ---
ignore_startup_parameters = extra_float_digits
max_prepared_statements = 100
server_reset_query = DISCARD ALL
server_reset_query_always = 0

; --- logging ---
logfile = /var/log/postgresql/pgbouncer.log
pidfile = /var/run/postgresql/pgbouncer.pid
log_connections = 1
log_disconnections = 1
log_pooler_errors = 1
stats_period = 60
```

Add the stats user (read-only monitoring) to userlist.txt in the same way as the others, or create it in PostgreSQL and rely on the auth query.

Step 3 - Parameter reference

Parameter	Meaning and guidance
[databases] entry	Maps the alias clients use to a real server. Extra options per entry: `pool_size, min_pool_size, reserve_pool, pool_mode, max_db_connections, connect_query, auth_user`.
listen_addr / listen_port	Interfaces and port to accept clients. Use a specific IP rather than * where possible.
auth_type	Use scram-sha-256. md5 only for legacy. `hba` lets pg_hba-style rules in auth_hba_file decide.
auth_file / auth_user / auth_query	Where credentials come from (file, or lookup in PostgreSQL via auth_user).
admin_users / stats_users	Who may use the admin console fully / read-only (SHOW commands).
pool_mode	session, transaction (recommended), or statement.
max_client_conn	Maximum client connections to PgBouncer. Can be large (thousands). Needs file descriptors.
default_pool_size	Server connections per (database, user) pair. Typical 10 to 50.
min_pool_size	Keeps this many server connections warm to absorb bursts.
reserve_pool_size / reserve_pool_timeout	Extra connections allowed when clients wait longer than the timeout (seconds).
max_db_connections	Hard cap on server connections for one database, across all users. Protects max_connections.
max_user_connections	Hard cap per user across databases.
server_idle_timeout	Close server connections idle longer than this (seconds).
server_lifetime	Recycle server connections after this age, so failover and memory growth are handled.
query_wait_timeout	Disconnect clients that wait for a server connection longer than this. Prevents infinite hangs. Default 120.
ignore_startup_parameters	Startup parameters PgBouncer ignores instead of rejecting (JDBC sends extra_float_digits).
max_prepared_statements	Number of prepared statements tracked per client for transaction mode. 0 disables. 1.21+.
server_reset_query	Run when a connection returns to the pool. Used in session mode (DISCARD ALL). Not run in transaction mode unless server_reset_query_always=1.
log_connections / log_disconnections	Useful for audits and debugging; very chatty on busy systems. Consider 0 in production.
stats_period	Seconds between statistics lines written to the log.

6. Start, Test and Connect Applications

Step 1 - Validate and start

```
sudo systemctl enable --now pgbouncer
sudo systemctl status pgbouncer
sudo tail -n 30 /var/log/postgresql/pgbouncer.log
ss -ltnp | grep 6432
```

Step 2 - Test a client connection through PgBouncer

```
psql -h 127.0.0.1 -p 6432 -U appuser -d appdb -c "SELECT current_user, inet_server_port();"
```

The password is the same as the PostgreSQL role password. Note that you must use the PgBouncer port (6432), not 5432.

Step 3 - Test the admin console

```
psql -h 127.0.0.1 -p 6432 -U pgbouncer -d pgbouncer
pgbouncer=# SHOW POOLS;
pgbouncer=# SHOW DATABASES;
```

Step 4 - Point the application to PgBouncer

Setting	Change
Host	PgBouncer host (or 127.0.0.1 if local)
Port	6432
Database	The alias defined in [databases] (for example appdb)
Driver pool size	Keep the application-side pool modest; PgBouncer multiplexes the rest
Prepared statements	Check Section 12 for your driver (JDBC, psycopg, node-postgres, etc.)

Step 5 - Roll out gradually

- Move one low-risk application or a fraction of instances first.
- Watch `SHOW POOLS` for waiting clients and the application error rate.
- Then move the rest and lower the applications' own connection pool sizes.

7. Sizing the Pools

The goal: total possible server connections stay safely below PostgreSQL's `max_connections`.

```
Pools are per (database, user) pair.

Max server connections = SUM over every (database, user) pair of ( pool_size + reserve_pool_size )

Requirement:   Max server connections + superuser_reserved_connections + admin/monitoring/replication
               connections  <=  max_connections
```

Worked example

Item	Value
PostgreSQL server	16 CPU cores, max_connections = 200
Reserved for superuser, monitoring, backups, replication	about 30
Available for PgBouncer	170
Pools in use	appdb with 2 users (appuser, reportuser)
Setting	default_pool_size = 30, reserve_pool_size = 5, max_db_connections = 100
Worst case	2 x 35 = 70, capped at 100 by max_db_connections: safe

Rules of thumb

- Active PostgreSQL work scales to roughly 2 to 4 times the CPU core count; more concurrent active backends rarely add throughput.
- Start with default_pool_size 20 to 30 for transaction mode and adjust using SHOW POOLS (watch cl_waiting and maxwait).
- Use `max_db_connections` as the safety net; it limits total server connections for a database even with many users.
- Several PgBouncer instances (one per app host) multiply pool usage: sum the limits of all instances.
- Slow queries hold connections. A bigger pool does not fix slow queries; investigate them first.
- Set `max_client_conn` well above expected clients (it is cheap), but ensure the file descriptor limit allows it.

8. Administration

Admin console

```
psql -h 127.0.0.1 -p 6432 -U pgbouncer -d pgbouncer
```

Command	Purpose
SHOW HELP / SHOW VERSION	List commands / show PgBouncer version.
SHOW CONFIG	All settings currently in effect.
SHOW DATABASES	Configured databases, host, port, pool size, current connections.

Command	Purpose
SHOW POOLS	Per pool client and server connection counts (most important view).
SHOW CLIENTS	Every client connection: user, database, state, address, wait time.
SHOW SERVERS	Every server connection: state, address, age, current activity.
SHOW STATS / SHOW STATS_TOTALS	Traffic and timing counters per database (rates or totals).
SHOW USERS	Configured users and their pool mode.
SHOW LISTS	Internal counters (databases, users, pools, free clients, used clients).
SHOW MEM / SHOW FDS / SHOW SOCKETS	Memory use, open file descriptors, sockets.
RELOAD	Re-read pgbouncer.ini and userlist.txt without disconnecting clients.
PAUSE [db] / RESUME [db]	Wait for active queries to finish and hold new ones; resume afterwards.
SUSPEND	Flush buffers and stop listening (for online restarts).
DISABLE db / ENABLE db	Reject or accept new connections to one database.
KILL db	Drop all client and server connections for a database.
RECONNECT [db]	Close server connections after current use so new ones are created (used after changing host).
SET name = value	Change a runtime setting temporarily (lost on restart unless added to the ini).
SHUTDOWN	Stop PgBouncer.

Procedure: apply a configuration change

```
sudo nano /etc/pgbouncer/pgbouncer.ini
sudo systemctl reload pgbouncer          # or: RELOAD; in the admin console
# Check: SHOW CONFIG;
```

Most settings take effect on reload. Changes to `listen_addr`, `listen_port` and some file/socket settings need a restart.

Procedure: PostgreSQL maintenance or backend change without dropping clients

```
-- 1. In admin console: hold new queries and wait for running ones
PAUSE appdb;
-- 2. Do the maintenance (restart PostgreSQL, switch host, etc.)
--    If the backend host changed: edit [databases] in pgbouncer.ini, then:
RELOAD;
RECONNECT appdb;
-- 3. Release the clients
RESUME appdb;
```

Procedure: add a new application user

- With auth_query: just create the role in PostgreSQL and add it to pg_hba.conf if needed. No PgBouncer change.

- With userlist.txt only: add the user's line to userlist.txt and run RELOAD.

Procedure: upgrade PgBouncer

```
sudo apt install --only-upgrade pgbouncer        # or dnf update pgbouncer
sudo systemctl restart pgbouncer                 # clients reconnect (brief interruption)
```

For zero-downtime upgrades, run two PgBouncer instances behind a load balancer and restart them one at a time.

9. Monitoring and Alerting

9.1 Admin console views

SHOW POOLS is the first place to look. Key columns:

Column	Meaning	What to watch
cl_active	Clients currently linked to a server connection	Normal activity
cl_waiting	Clients waiting for a server connection	Should be 0 most of the time. Sustained values mean the pool is saturated.
sv_active	Server connections in use	Close to pool_size means saturation
sv_idle	Server connections ready for reuse	Healthy buffer
sv_used	Idle connections that need a reset check before reuse	Normal
sv_login	Server connections currently logging in	High values suggest connection storms or slow auth
maxwait (and maxwait_us)	Age in seconds of the oldest waiting client	Alert when above 1 to 5 seconds consistently
pool_mode	Pool mode in effect	Verify it matches your design

SHOW STATS columns: `total_xact_count`, `total_query_count`, `total_received`, `total_sent`, `total_xact_time`, `total_query_time`, `total_wait_time` and per-second averages (`avg_xact_count`, `avg_query_count`, `avg_recv`, `avg_sent`, `avg_xact_time`, `avg_query_time`, `avg_wait_time`). A growing **avg_wait_time** means clients are spending time queueing for a server connection.

```
SHOW POOLS;
SHOW STATS;
SHOW CLIENTS;       -- who is waiting
SHOW SERVERS;       -- what each server connection is doing
SHOW LISTS;
```

9.2 PgBouncer logs

With `stats_period = 60`, PgBouncer writes a summary line every minute (requests per second, bytes in/out, average transaction and query time, average wait time). Also watch for these message types:

- **Errors and warnings** such as pooler errors, authentication failures and server connection problems.
- **closing because: client_idle_timeout / query_wait_timeout** - clients disconnected due to timeouts.

- **no more connections allowed** - a configured limit was reached.

```
sudo tail -f /var/log/postgresql/pgbouncer.log
sudo grep -E "ERROR|WARNING|no more connections" /var/log/postgresql/pgbouncer.log | tail -n 50
```

9.3 Prometheus and Grafana (recommended for production)

Use the community **pgbouncer_exporter** to expose the admin console statistics to Prometheus.

```
# Install the exporter (download a release from github.com/prometheus-community/pgbouncer_exporter)
# Create a dedicated, minimal stats user in userlist.txt / PostgreSQL (listed in stats_users)

pgbouncer_exporter \
  --pgBouncer.connectionString="postgres://pgbouncer_stats:password@127.0.0.1:6432/pgbouncer?sslmode=dis
  --web.listen-address=":9127"
```

- Prometheus scrape job: target *pgbouncer-host:9127*.
- Import a PgBouncer dashboard in Grafana or build panels for waiting clients, active/idle server connections, query and transaction rates and wait time.
- Run the exporter as a systemd service under a non-root user. Check `pgbouncer_exporter --help` for exact flags in your version.

9.4 Checks on the PostgreSQL side

```
-- Connections by user and state (should now be small and stable)
SELECT usename, application_name, state, count(*)
FROM pg_stat_activity
WHERE backend_type = 'client backend'
GROUP BY 1,2,3 ORDER BY 4 DESC;

-- Headroom against max_connections
SELECT count(*) AS used, current_setting('max_connections')::int AS max FROM pg_stat_activity;

-- Long transactions that hold pooled connections
SELECT pid, usename, now()-xact_start AS xact_age, state, left(query,80)
FROM pg_stat_activity WHERE xact_start IS NOT NULL ORDER BY xact_start LIMIT 10;
```

9.5 Suggested alerts

Alert	Condition (tune to your workload)	Action
PgBouncer down	Process or port 6432 not responding	Restart service; check logs
Pool saturation	cl_waiting greater than 0 for more than 30 to 60 seconds	Check slow queries, raise pool size if PostgreSQL has capacity
Long queue wait	maxwait above 5 seconds, or avg_wait_time rising	As above; look for long transactions
Server pool near limit	sv_active at 90 percent of pool_size for several minutes	Review sizing (Section 7)
Client limit near	Client connections above 80 percent of max_client_conn	Raise max_client_conn and file descriptor limit
Auth failures spike	Repeated authentication errors in the log	Check passwords, pg_hba.conf, brute force
PostgreSQL connections high	pg_stat_activity count above 80 percent of max_connections	Reduce pool sizes or add pooler caps
Exporter or scrape missing	No metrics for 2 minutes	Check exporter and network

10. Security

TLS from clients to PgBouncer

```
client_tls_sslmode = require            ; or verify-full when clients use certificates
client_tls_key_file = /etc/pgbouncer/server.key
client_tls_cert_file = /etc/pgbouncer/server.crt
client_tls_ca_file = /etc/pgbouncer/ca.crt
```

TLS from PgBouncer to PostgreSQL

```
server_tls_sslmode = verify-full
server_tls_ca_file = /etc/pgbouncer/ca.crt
```

PostgreSQL must have `ssl = on` with a certificate PgBouncer trusts. Set file ownership to the PgBouncer OS user and key permissions to 600.

Client access rules (pg_hba style) inside PgBouncer

```
auth_type = hba
auth_hba_file = /etc/pgbouncer/pgbouncer_hba.conf

# pgbouncer_hba.conf
host    appdb    appuser    10.0.1.0/24    scram-sha-256
host    all      all        0.0.0.0/0      reject
local   pgbouncer  pgbouncer              scram-sha-256
```

Hardening checklist

- Restrict port 6432 with a firewall to application hosts only. Never expose it to the internet.
- userlist.txt and key files: owned by the PgBouncer OS user, mode 640 or 600.
- Use SCRAM; avoid plain-text passwords. Use a dedicated low-privilege `pgbouncer` role for auth_query.
- Limit `admin_users` to DBAs; give monitoring tools `stats_users` only.
- Do not run PgBouncer as root.
- Keep PgBouncer updated for security fixes.
- Be careful with log_connections in sensitive environments; logs contain user, database and IP data.

11. High Availability and Failover

A single PgBouncer is a single point of failure. Common designs:

Design	Description	Notes
PgBouncer on each application host	Apps connect to localhost:6432	No extra network hop; no central pooler to fail. Remember pool totals multiply per host.
Two PgBouncers behind HAProxy / keepalived VIP	Apps use the VIP; one instance can restart without downtime	Common central design. Use health checks on port 6432.
PgBouncer in Kubernetes	Deployment with several replicas behind a Service	Set pool sizes per pod so the total stays within max_connections.

Working with PostgreSQL failover (Patroni, repmgr, manual)

- Point the [databases] entry at a stable endpoint: the Patroni HAProxy/VIP, a DNS name for the primary, or a service address that always follows the primary.
- With a DNS-based endpoint, set a reasonable `dns_max_ttl` and a short `server_lifetime` so stale connections are replaced.
- After a switchover, existing server connections to the old primary fail. PgBouncer discards them and opens new ones; clients may see a few errors, so applications need retry logic.
- If the host in [databases] is changed by a script: edit the ini, `RELOAD;` then `RECONNECT dbname;` (or use PAUSE / RESUME around the switch).
- Use a separate alias (appdb_ro) pointing to a replica or read-only load balancer for read traffic.

Planned PostgreSQL switchover with minimal errors

```
-- 1. PAUSE appdb;                  (admin console; waits for in-flight queries)
-- 2. Perform switchover / promote the new primary
-- 3. Update the host if needed in pgbouncer.ini, then RELOAD; RECONNECT appdb;
-- 4. RESUME appdb;
```

12. Application Compatibility in Transaction Mode

Driver / tool	Recommended action
JDBC (Java)	Version 42.x: add `prepareThreshold=0` if PgBouncer is older than 1.21 or max_prepared_statements=0. With 1.21+ and max_prepared_statements greater than 0, prepared statements work. Keep `ignore_startup_parameters = extra_float_digits`.
psycopg 3 (Python)	Set `prepare_threshold=None` on connections for older PgBouncer. With 1.21+ and max_prepared_statements, automatic prepares work.
psycopg2 / SQLAlchemy	No server-side prepared statements by default; usually fine. Do not use session-level SET without SET LOCAL.
node-postgres (pg)	Avoid named prepared statements on older PgBouncer; plain parameterized queries are fine.
Go (pgx)	Use `default_query_exec_mode=exec` or simple protocol on older PgBouncer.
.NET (Npgsql)	Set `No Reset On Close=true` and review auto-prepare settings. Prefer Multiplexing off with PgBouncer.
Migration tools and pg_dump	Connect directly to PostgreSQL on 5432 (or via a session-mode alias).
Frameworks using advisory locks (e.g. migration runners)	Use session mode alias, or transaction-level advisory locks.

Note: Always test the full application, including batch jobs and migrations, against the PgBouncer endpoint in a staging environment before cutover. Driver option names can differ by version; confirm in your driver's documentation.

13. Troubleshooting and Cheat Sheet

Symptom / log message	Likely cause and fix
no more connections allowed (max_client_conn)	Client limit reached. Raise max_client_conn and the open files limit; find clients that leak connections.
no more connections allowed (max_db_connections) or pool full; clients stuck waiting	Pool saturated. Check SHOW POOLS (cl_waiting), long transactions, slow queries; adjust default_pool_size / max_db_connections within max_connections.
password authentication failed	Wrong password, wrong hash in userlist.txt, role has MD5 hash with auth_type=scram-sha-256, or pg_hba.conf rejects PgBouncer's IP.
no such user / bouncer config error	User missing from userlist.txt and auth_query not working. Test the auth function as the pgbouncer role.
auth_query failed / permission denied for function	GRANT EXECUTE not applied, function missing in that database, or set auth_dbname.
unsupported startup parameter: extra_float_digits	Add it to ignore_startup_parameters (also options if the driver sends it).
prepared statement "..." does not exist	Server-side prepared statements in transaction mode. Upgrade to 1.21+ and set max_prepared_statements, or disable prepares in the driver.
Settings seem lost between queries (SET, temp tables, advisory locks)	Transaction pooling. Use SET LOCAL, transaction-level locks, or a session-mode alias.
query_wait_timeout	Clients waited too long for a server connection. Investigate saturation or increase pool/timeouts.
server conn crashed? / closing because: server lifetime	Backend restarted or terminated, or lifetime recycling. Check the PostgreSQL log.
Too many open files / accept() failed	Raise LimitNOFILE in the systemd override and restart.
SSL errors	Check certificate paths, permissions, sslmode on both sides, and that PostgreSQL ssl=on.

Command cheat sheet

Task	Command
Start / stop / reload	systemctl start|stop|reload pgbouncer
Admin console	psql -h 127.0.0.1 -p 6432 -U pgbouncer pgbouncer
Pool status	SHOW POOLS;
Traffic statistics	SHOW STATS;
Who is connected	SHOW CLIENTS; SHOW SERVERS;
Apply config	RELOAD;
Hold / release queries	PAUSE appdb; RESUME appdb;
Refresh server connections	RECONNECT appdb;
Test through pooler	psql -h HOST -p 6432 -U appuser -d appdb
Tail logs	tail -f /var/log/postgresql/pgbouncer.log
PostgreSQL connection count	SELECT count(*) FROM pg_stat_activity;

Go-live checklist

- PostgreSQL: auth role and function created, pg_hba.conf updated, max_connections headroom confirmed.
- PgBouncer: userlist.txt secured, pool sizes calculated, timeouts set, file descriptor limit raised.
- Application: tested in transaction mode, prepared statements and session features reviewed, retry logic in place.
- Monitoring: SHOW POOLS alerts, exporter and dashboards, log alerts.
- HA: second instance or per-host deployment decided; failover procedure documented and tested.
- Backups and tools (pg_dump, pgBackRest, replication) keep using direct PostgreSQL connections.

References: pgbouncer.org/config.html, pgbouncer.org/usage.html, pgbouncer.org/features.html